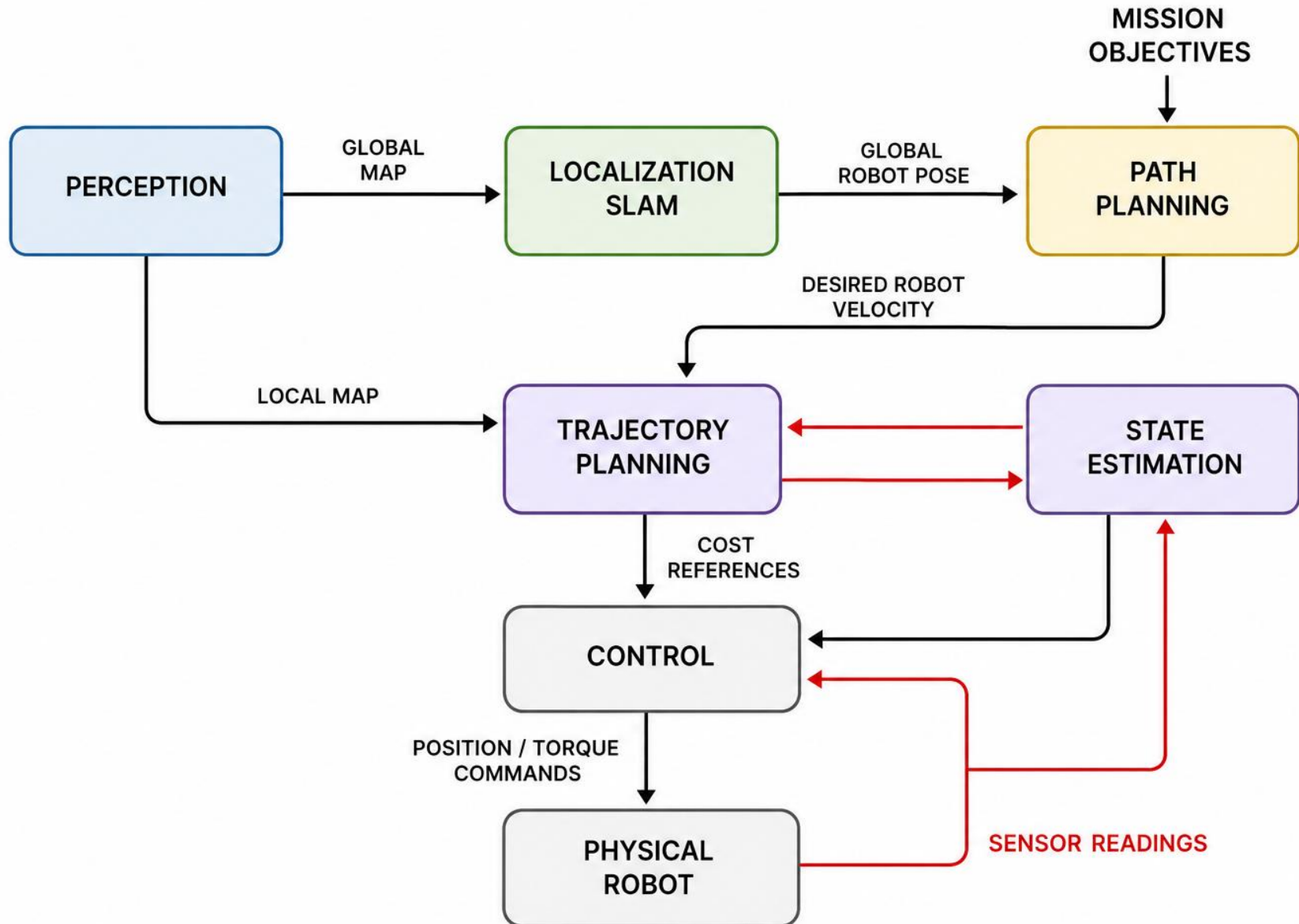


Lower Dimensional Models

Prof. Michele Focchi, Associate Professor
University of Trento

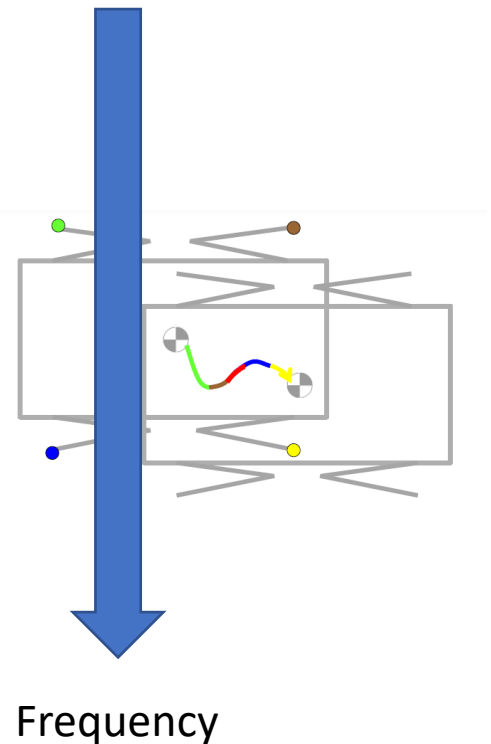


Typical Robot Control Framework



Key problems in legged robotics

- Localization (Where the robot is in a map of the environment)
- Path planning (how to move the robot avoiding obstacles)
- Trajectory planning (com and joint trajectories: how to move each limb)
- Perception (vision to understand the environment)
- Estimation (what is the state of the robot)
- Control (how to apply forces to the joints)



Outline

Limits of Instantaneous Control

Single Rigid Body Model

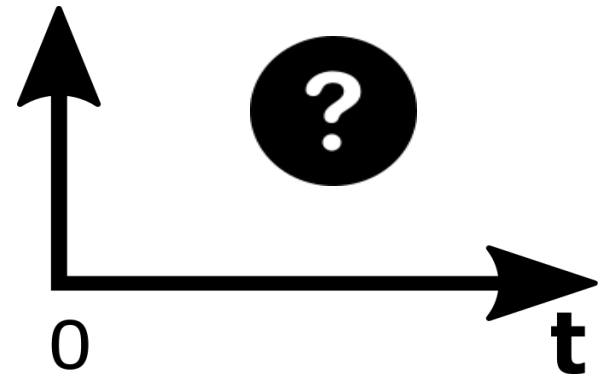
Centroidal Model

Linear Inverted Pendulum Model (LIPM)

Other Models

Classic (Instantaneous) Reactive Control

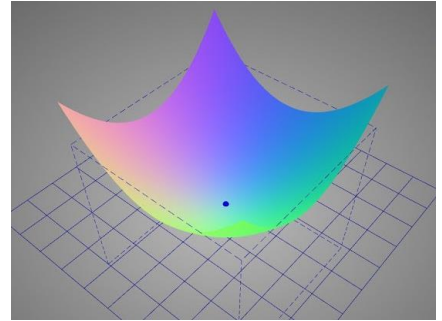
- need reference trajectory to track or a state to stabilize
- Good enough for industrial settings in which you know the trajectory the robot should track
- Instantaneous control is myopic: **has no awareness of future**



- Can handle constraints in the actual instant (torque/velocity limits)
- Difficult to handle constraint in the future (e.g. joint positions/collision avoidance)
- Need to be able to look in the future to take action now to avoid violating later

Trajectory optimization/optimal control

- No need of a reference trajectory (optional)
- Objective clearly specified through cost function
- Using models we can predict the future in order to fulfill constraints
- Accounts for future constraints in current decisions
- A solver will find a motion that minimizes the cost function and fulfill constraints
- However... for legger robots



C3 Many Degrees of Freedom

C1 Heavily constrained



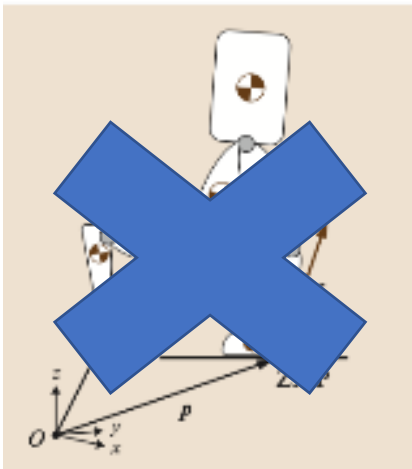
Optimization for legged robots is
Computationally **expensive**!
Computation time explodes with
number of states / constraints

Lower dimensional models motivations

- Models are intrinsically inaccurate
- We want to quickly recover from disturbance, terrain change
- With legged robots we need to react quickly because the dynamics is **unstable**
- We want accommodate to velocity set-point set by the user
- multi-body free floating complete model is too complex

re-planning

Reduce
Model complexity



Introduce Lower dimensional models

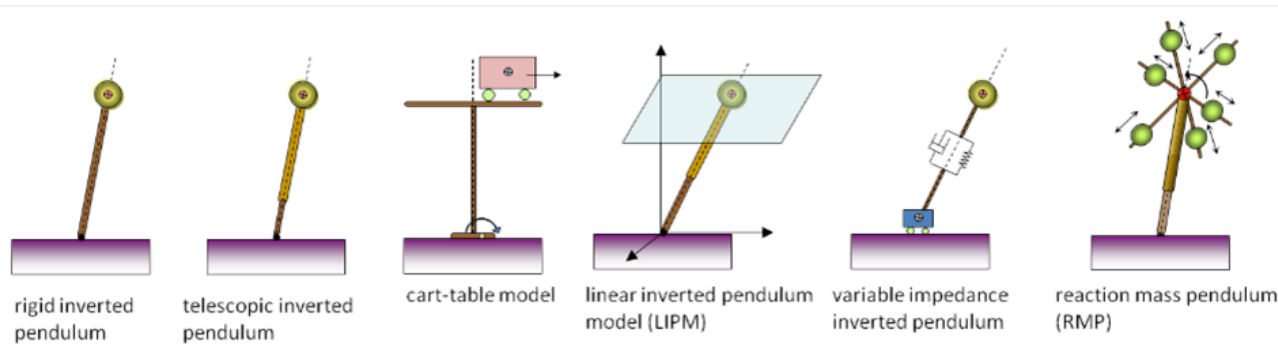


Optimization is faster (can then be carried out **online**)

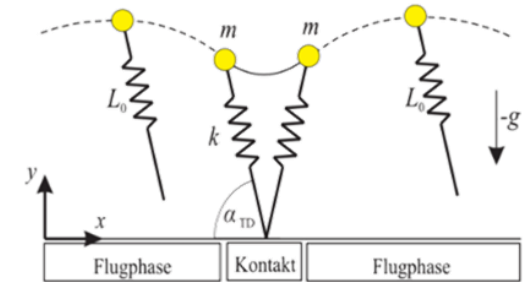


trade-off between **accuracy** and **computation time**

Lower dimensional models types



for walking/balancing



for running

Lower dimensional models features:

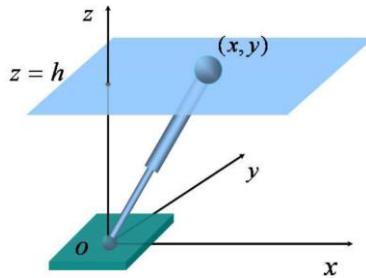
- Capture only the predominant dynamics of the robot
- Limited number of states
- Linear  More tractable from the Numerical point of view
- Non linear (but easy to be convexified)

Locomotion Recipe:

- ① Optimize the trajectory with lower dimensional model
- ② Use a controller to maps from dominant dynamics (e.g. CoM dynamics for walking) to the complete state (torques).

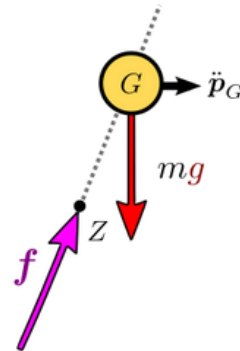
Lower dimensional models types

- We will cover in detail the most popular ones



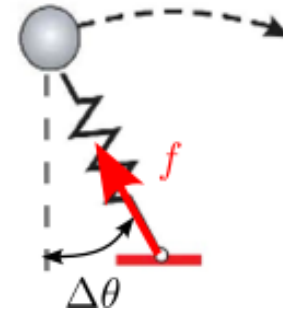
Linear inverted pendulum - LIP (2)

INPUT: ZMP



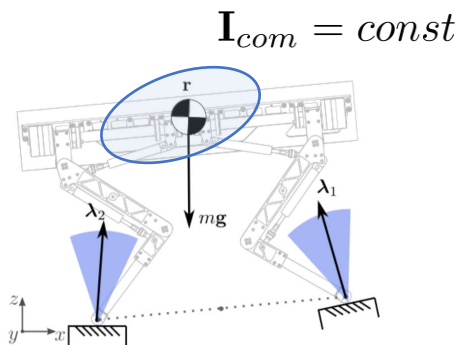
Point mass model – PM (3)

INPUT: force through CoM



Spring loaded inv. pendulum - SLIP (3)

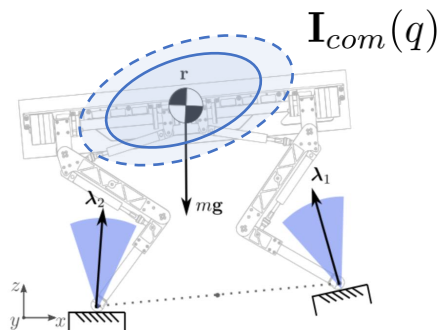
INPUT: Leg force or thrust, Leg placement adjustment ($\Delta\theta$):



Single Rigid Body Dyn. – SRBD (6)

INPUT: Contact forces

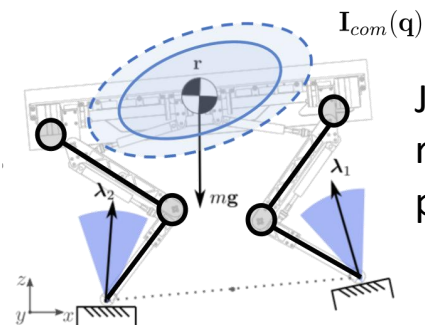
- constant inertia



Centroidal Dynamics –CD (6)

INPUT: Contact forces

- Variable inertia due to joint motion



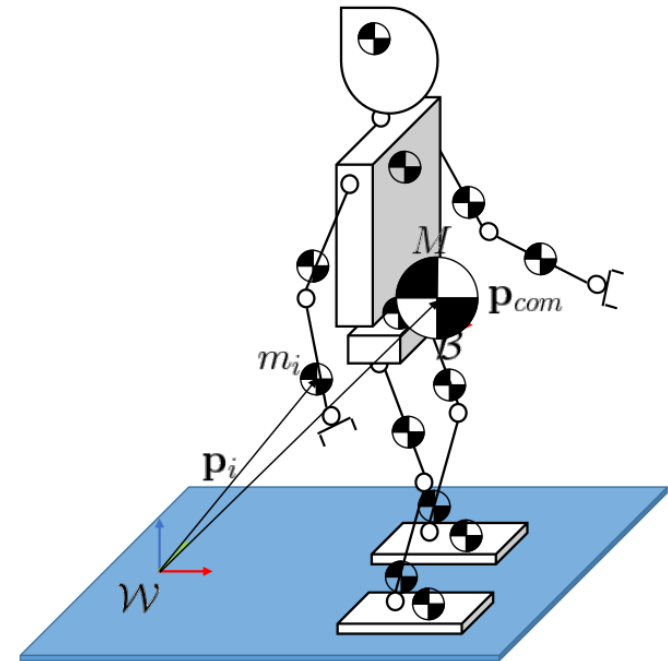
Centroidal Dyn. + Full kin.(6+n)

INPUT: Contact forces, joint velocities v_j

Full floating base dynamics (recap)

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{S}^T \boldsymbol{\tau} + \mathbf{J}(\mathbf{q})^T \mathbf{f}$$

- models **both** floating base (linear and angular) and leg joints dynamics
 - allows to implement **friction constraints**, **joint kinematic limits** and **torque constraints**
 - INPUTS**: joint torques τ
-
- Note: Henceforth we will always assume **point-contacts**



Outline

Limits of Instantaneous Control

Single Rigid Body Model

Centroidal Model

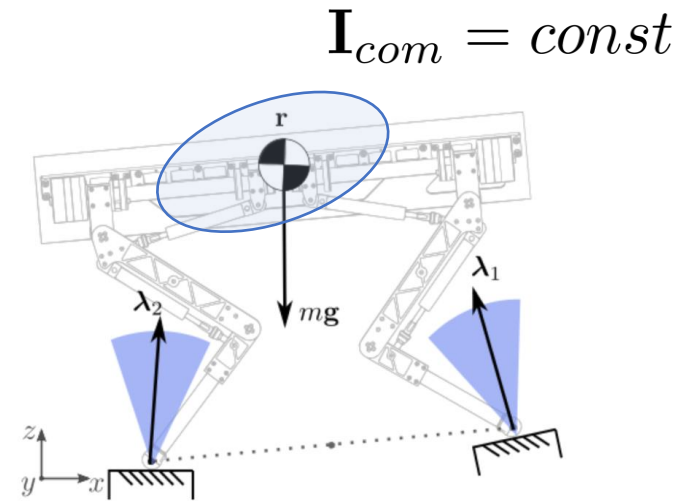
Linear Inverted Pendulum Model (LIPM)

Other Models

Single Rigid Body Dynamics

Assumption 1 (A1): Leg dynamics is neglected

- ✓ removed dependency on joint angles
- ✓ Their position no longer influences robot inertia
- ✗ No joint limits / torque limits



Assumption 2 (A2): Robot inertia L_G is considered as the one of a **single rigid body** and is **constant** to the one of the robot in a nominal joint position (i.e. neglect “gesticulation”)

- Dynamics described by Newton-Euler equations

- ✓ Captures **linear** and **angular** dynamics

Only responsible of angular dynamics

- **INPUTS:** contact forces f

- ✓ friction limits are still expressable

- **non** coplanar contacts

- ✓ Enables walk on uneven terrains!

- ✗ Non linearity

$$m\ddot{x}_c = mg + \sum_{i=1}^{n_c} f_i$$

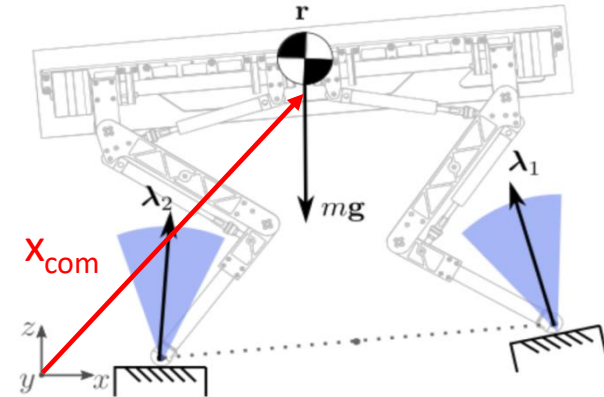
$$I_{com}\dot{\omega} + \omega \times I_{com}\omega = \sum_{i=1}^{n_c} (x_{f,i} - x_{com}) \times f_i$$



Basic Assumption: the robot has enough joint torques to realize the actuated part of the equation

Single Rigid Body Dynamics – state space form

- **STATE:** base pose (position + orientation)+base twist
- **INPUT:** contact forces f
- **Parameters:** feet positions
- Mass of the legs negligible compared to the base $\bar{\omega} \simeq \omega_b$



State Space Procedure

- ① Put in explicit matrix form and write Euler equation about a generic frame not attached to CoM (e.g world)

$$\underbrace{\begin{bmatrix} mI_{3 \times 3} & 0_{3 \times 3} \\ m[x_{\text{com}}]_{\times} & I_{\text{com}} \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} \ddot{x}_{\text{com}} \\ \dot{\omega} \end{bmatrix}}_{\mathbf{X}} = \underbrace{\begin{bmatrix} I_{3 \times 3} & \cdots & I_{3 \times 3} \\ [x_{f_1}]_{\times} & \cdots & [x_{f_c}]_{\times} \end{bmatrix}}_{\mathbf{U}} \underbrace{\begin{bmatrix} f_1 \\ \vdots \\ f_c \end{bmatrix}}_{\mathbf{u}} + \begin{bmatrix} mg \\ x_{\text{com}} \times mg \end{bmatrix} + \begin{bmatrix} 0 \\ -\omega \times I_{\text{com}} \omega \end{bmatrix}$$

- ② Define state/input vectors

$$\mathbf{x} = [\mathbf{x}_{\text{com}}, \Phi, \dot{\mathbf{x}}_{\text{com}}, \boldsymbol{\omega}] \in \mathbb{R}^{12}$$

$$h_O = x_{\text{com}} \times m\dot{x}_{\text{com}} + I_{\text{com}}\omega$$

$$\frac{d}{dt}(x_{\text{com}} \times m\dot{x}_{\text{com}}) = \underbrace{x_{\text{com}} \times m\ddot{x}_{\text{com}}}_{\text{red circle}} + \underbrace{\dot{x}_{\text{com}} \times m\dot{x}_{\text{com}}}_0$$

$$\mathbf{u} = [\mathbf{f}_1 \quad \dots \quad \mathbf{f}_c] \in \mathbb{R}^{3c} \quad \Rightarrow \quad \text{Grows with number of contacts}$$

Single Rigid Body Dynamics – state space form

③ Put in state-space form (assuming Euler angles Φ to parametrize orientation)

$$\dot{\mathbf{x}} = \underbrace{\begin{bmatrix} \dot{\mathbf{x}}_{com} \\ \mathbf{E}(\Phi)\boldsymbol{\omega} \\ \mathbf{g} \\ -\mathbf{I}_{com}^{-1} [\boldsymbol{\omega} \times (\mathbf{I}_{com}\boldsymbol{\omega})] \end{bmatrix}}_{\mathbf{f}(\mathbf{x})} + \underbrace{\begin{bmatrix} \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \cdots & \mathbf{0} \\ \frac{1}{m}\mathbf{I}_3 & \cdots & \frac{1}{m}\mathbf{I}_3 \\ I_{com}^{-1} [\mathbf{p}_{f,1} - \mathbf{x}_{com}]_{\times} & \cdots & I_{com}^{-1} [\mathbf{p}_{f,c} - \mathbf{x}_{com}]_{\times} \end{bmatrix}}_{\mathbf{g}(\mathbf{x})} \begin{bmatrix} \mathbf{f}_1 \\ \vdots \\ \mathbf{f}_c \end{bmatrix}$$

Mapping between angular velocity and Euler rates

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u} \quad \text{nonlinear and control-affine}$$

Outline

Limits of Instantaneous Control

Single Rigid Body Model

Centroidal Dynamics Model

Linear Inverted Pendulum Model (LIPM)

Other Models

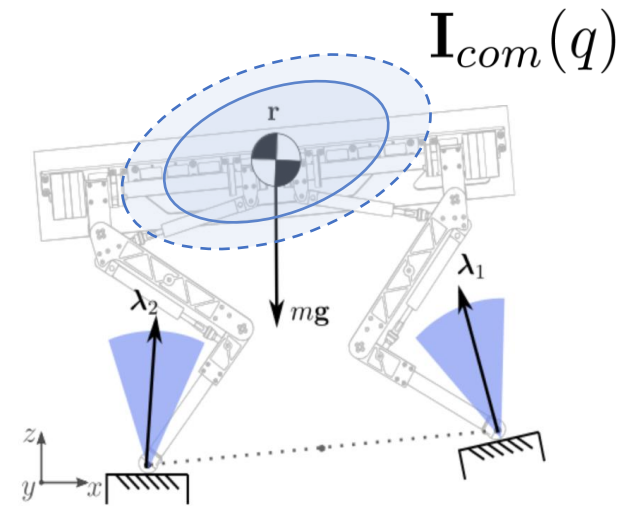
Centroidal Dynamics Model

- I_{com} changes because of joint motion
- Let's set angular momentum as a state instead of ω

$$\dot{\mathbf{x}}_{com} = \frac{1}{m} \mathbf{l},$$

$$\dot{\mathbf{l}} = m\mathbf{g} + \sum_{i=1}^{n_c} \mathbf{f}_i,$$

$$\dot{\mathbf{k}}_G = \sum_i^{n_c} (\mathbf{x}_{f,i} - \mathbf{x}_{com}) \times \mathbf{f}_i$$



[Herzog et al 2015.]

[Ponton et al. 2020]

STATE:

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_{com} \\ \mathbf{l} \\ \mathbf{k}_G \end{bmatrix}$$

$\in \mathbb{R}^9$

these are the quantities whose evolution is determined directly by the contact forces

Linear momentum ($\cong \dot{\mathbf{x}}_{com}$)

Angular momentum

INPUT:

$$\mathbf{u} = \begin{bmatrix} \mathbf{f}_1 \\ \vdots \\ \mathbf{f}_c \end{bmatrix} \in \mathbb{R}^{3c}$$



No body orientation, angular velocity or inertia is needed, no gyroscopic term.



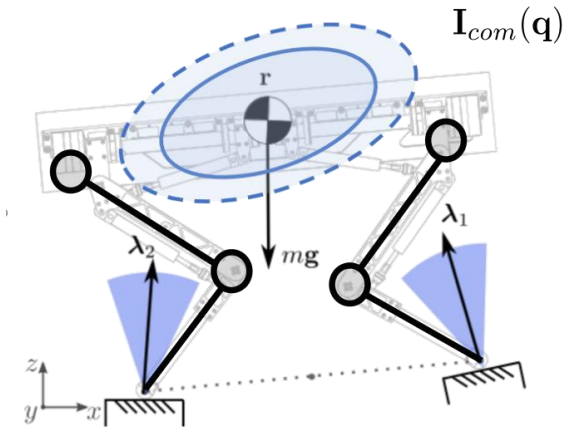
contact forces directly determine $\dot{\mathbf{k}}_G$, so the model can prescribe a trajectory for $\mathbf{k}_G$, but it cannot determine how the base and joints must move to generate it.



Non linearity

Centroidal Dynamics + full kinematics

- Centroidal dynamics describe the evolution of the robot's total momentum under external forces
- Full-body kinematics connect the centroidal momentum to the floating-base and joint motion.



The generalized configuration and velocity are

$$\mathbf{q} = \begin{bmatrix} \mathbf{q}_b \\ \mathbf{q}_j \end{bmatrix}, \quad \dot{\mathbf{q}} = \begin{bmatrix} \dot{\mathbf{q}}_b \\ \dot{\mathbf{q}}_j \end{bmatrix}, \quad \dot{\mathbf{q}}_b = \begin{bmatrix} \dot{\mathbf{x}}_{com} \\ \boldsymbol{\omega}_B \end{bmatrix}.$$

- The centroidal momentum is:

$$\mathbf{h}_G = \begin{bmatrix} \mathbf{l} \\ \mathbf{k}_G \end{bmatrix} = \mathbf{A}_G(\mathbf{q})\dot{\mathbf{q}},$$

where $\mathbf{l}$ is the linear momentum, $\mathbf{k}_G$ is the angular momentum about the CoM, and $\mathbf{A}_G(\mathbf{q})$ is the Centroidal Momentum Matrix (CMM).

Centroidal Dynamics + full kinematics

- Partitioning the Centroidal Momentum Matrix into floating-base and joint contributions gives

$$\mathbf{h}_G = [\mathbf{A}_B(\mathbf{q}) \quad \mathbf{A}_j(\mathbf{q})] \begin{bmatrix} \dot{\mathbf{q}}_b \\ \dot{\mathbf{q}}_j \end{bmatrix} = \mathbf{A}_b(\mathbf{q})\dot{\mathbf{q}}_b + \mathbf{A}_j(\mathbf{q})\dot{\mathbf{q}}_j.$$

- Therefore, given the centroidal momentum ($\mathbf{l}$, $\mathbf{k}_G$) and the joint velocities, the floating-base velocity is

$$\dot{\mathbf{q}}_b = \mathbf{A}_b^{-1}(\mathbf{q}) (\mathbf{h}_G - \mathbf{A}_j(\mathbf{q})\dot{\mathbf{q}}_j).$$

- We have seen by expressing the change of momentum in a frame anchored at the current Center of Mass (also called centroid) that we can rewrite the full dynamics as

$$\dot{\mathbf{h}}_G = \mathbf{A}(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{A}}(\mathbf{q})\dot{\mathbf{q}} = \begin{bmatrix} m\mathbf{g} + \sum_{i=1}^{n_i} \mathbf{f}_i \\ \sum_{i=1}^{n_i} \mathbf{x}_{f,i}(\mathbf{q}) \times (\mathbf{x}_{com}(\mathbf{q}) \times \mathbf{f}_i) \end{bmatrix}$$





Rate of change of momentum

$\mathbf{A} \in \mathbb{R}^{6 \times (6+n)}$ CMM maps the velocities, and therefore momentum, of each individual link into a common reference frame, expressed at CoM

Sum of ext. forces/moments

- These six equations relate the change of linear and angular momentum of all rigid bodies projected into the CoM with the external forces.



There is dependency on the joint states (additional non linearity)

State space model

STATE

$$\mathbf{x} = \begin{bmatrix} \mathbf{q}_b \\ \mathbf{h}_G \\ \mathbf{q}_j \end{bmatrix} \in \mathbb{R}^{12+n}$$

INPUT

$$\mathbf{u} = \begin{bmatrix} \mathbf{f}_1 \\ \vdots \\ \mathbf{f}_c \\ \mathbf{u}_j \end{bmatrix} \in \mathbb{R}^{3c+n}$$

- The state equations are

$$\dot{\mathbf{q}}_b = \mathbf{A}_B^{-1}(\mathbf{q}) [\mathbf{h}_G - \mathbf{A}_j(\mathbf{q})\mathbf{u}_j],$$

$$\dot{\mathbf{h}}_G = \begin{bmatrix} m\mathbf{g} + \sum_{i=1}^c \mathbf{f}_i \\ \sum_{i=1}^c [(\mathbf{x}_{f,i}(\mathbf{q}) - \mathbf{x}_{com}(\mathbf{q})) \times \mathbf{f}_i] \end{bmatrix},$$

$$\dot{\mathbf{q}}_j = \mathbf{u}_j, \quad \longrightarrow$$

joint velocities are treated as kinematic **inputs** and the joint positions are evolving as pure integrators (linear dynamics).



We can implement kinematics limits!



We can tell if the planned momentum trajectory is realizable by the robot's joints.



cross products still introduce nonlinearity + forward kinematics (cannot be handled with convex optimization)

Planning with Kino-Dynamic model

[Grandia et al. Perceptive Locomotion through Nonlinear Model Predictive Control, 2022]



Outline

Limits of Instantaneous Control

Single Rigid Body Model

Centroidal Dynamics Model

Linear Inverted Pendulum Model (LIPM)

Other Models

Linear inverted pendulum model

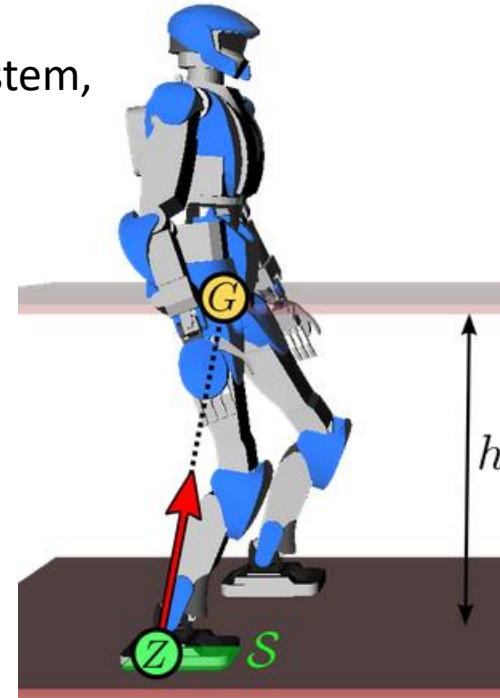
[Kajita et al. (2001)]

- Even in SRBD the angular momentum makes centroidal dynamics **non linear**.
- This means for instance that, to generate a trajectory for this system, one needs to solve a non linear optimization.
- We can **linearize** the SRBD dynamics making few assumptions:

Assumption A1: CoM height h is constant $\ddot{c}_z = \dot{c}_z = 0$

Assumption A2: no angular momentum/dynamics around the center of mass

Assumption A3: feet are coplanar (e.g. at constant height p_z)



- the robot can be seen as a point-mass concentrated at G resting on a mass-less leg in contact with the ground at a location Z
- we replace the contact forces with the resultant and define its application point CoP/ZMP as input to the system

Linear inverted pendulum model

[Winkler PhD Thesis (Ch. A.2), 2017]

$$\text{(SRBD)} \quad \dot{\mathbf{h}}_G \stackrel{(1)}{=} \begin{cases} m\ddot{\mathbf{c}} = m\mathbf{g} + \sum \mathbf{f}_i \\ (2) \quad \dot{\mathbf{k}}_G = \sum (\mathbf{p}_i - \mathbf{c}) \times \mathbf{f}_i \end{cases}$$

$$a \times b = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} \times \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix} = \begin{bmatrix} a_y b_z - a_z b_y \\ -a_x b_z + a_z b_x \\ a_x b_y - a_y b_x \end{bmatrix}$$

$$\text{(A2)} \quad \dot{\mathbf{k}}_G = 0 \quad \begin{cases} \dot{k}_x = 0 = \sum (p_{iy} - c_y) f_{iz} - \sum (p_{iz} - c_z) f_{iy} \\ \dot{k}_y = 0 = -\sum (p_{ix} - c_x) f_{iz} + \sum (p_{iz} - c_z) f_{ix} \end{cases} \Rightarrow 0 \quad \text{(A3)}$$

- Group terms depending on c to the left hand side

$$\begin{cases} c_y \sum f_{iz} - c_z \sum f_{iy} = \sum p_{iy} f_{iz} \\ -c_x \sum f_{iz} + c_z \sum f_{ix} = -\sum p_{ix} f_{iz} \end{cases}$$

- Divide by $\sum f_{iz}$

$$\begin{cases} c_y - c_z \frac{\sum f_{iy}}{\sum f_{iz}} = \frac{\sum p_{iy} f_{iz}}{\sum f_{iz}} \\ -c_x + c_z \frac{\sum f_{ix}}{\sum f_{iz}} = \frac{-\sum p_{ix} f_{iz}}{\sum f_{iz}} \end{cases}$$

Linear inverted pendulum model

$$(A1)+(1) \quad \ddot{c}_z = \dot{c}_z = 0 \quad \longrightarrow \quad 0 = m\ddot{c}_z = -mg + \sum f_{i,z} \quad \rightarrow \quad \boxed{\sum f_{i,z} = mg}$$

- Considering x,y components from (1)
- Why -? Scalar! $\mathbf{g} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix}$

$$\sum f_{i,x} = m\ddot{c}_x \quad \sum f_{i,y} = m\ddot{c}_y$$

- Putting all together

$$\begin{cases} c_y - c_z \frac{m\ddot{c}_y}{mg} = \frac{\sum p_{iy} f_{iz}}{\sum f_{iz}} \triangleq z_y \\ c_x - c_z \frac{m\ddot{c}_x}{mg} = \frac{\sum p_{ix} f_{iz}}{\sum f_{iz}} \triangleq z_x \end{cases}$$

Definition of center of pressure of contact forces

Robot height: Since it is constant it has no dynamics it will not be part of the state (only 2D)

Linear inverted pendulum Dynamics

- rearranging

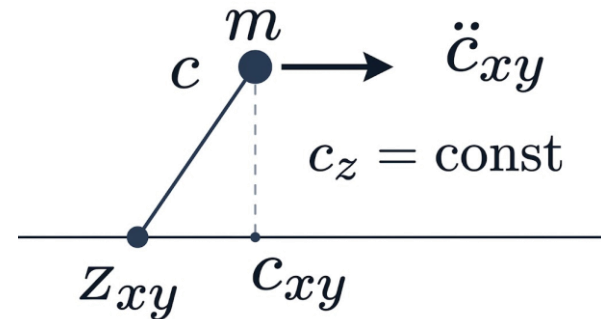
$$(1) \quad \ddot{c}_{xy} = \frac{g}{c_z} (c_{xy} - z_{xy}) \in \mathbb{R}^2$$

STATE

$$\mathbf{x} = \begin{bmatrix} c_x \\ c_y \end{bmatrix}$$

INPUT

$$\mathbf{u} = \begin{bmatrix} z_x \\ z_y \end{bmatrix}$$



CoM(c): point where inertia and gravity force originate

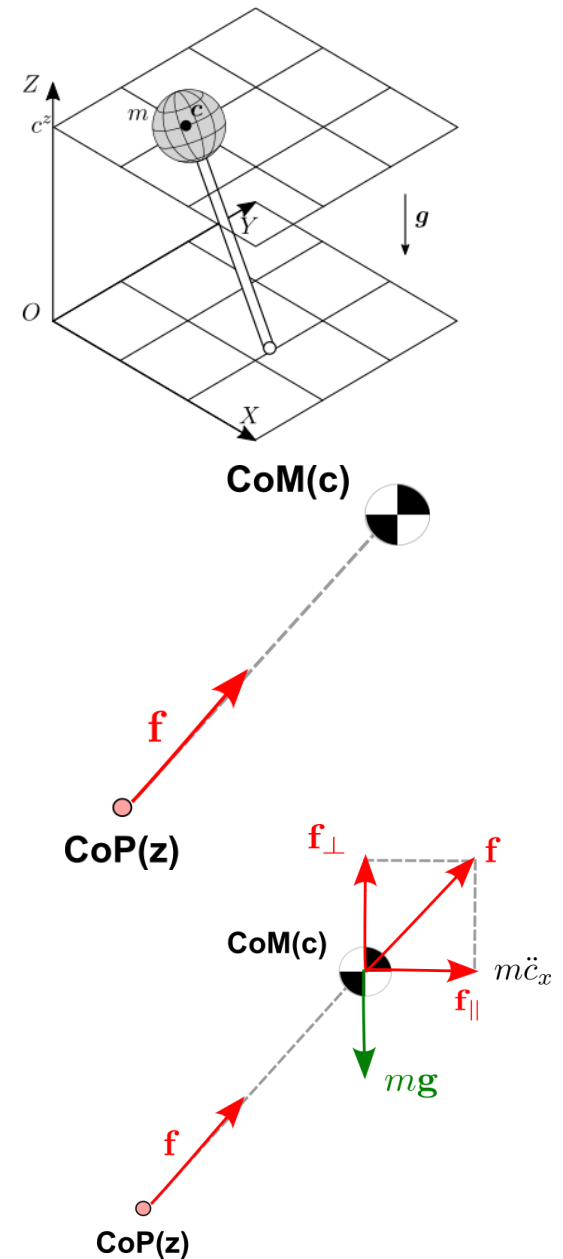
CoP(z): point where resultant of contact forces acts

- ✓ Small state (2D), **linear** dynamics
- ✗ Loses critical information about contact forces → cannot encode friction constraints
- ✗ No ang./vertical Dynamics → for more complex motions it can be essential to reorient the body/accelerate vertically (e.g. to jump)
- ✗ Limited to flat terrain => cannot place feet at different heights to cross uneven terrain.
- ✗ No joint constraints (torque/kinematic)

CoM dynamics for LIP: observations

- **Obs1:** LIP Dynamics (1) is linear and **decoupled** for x and y directions
- **Obs2:** If there is no moment about CoM (A2) the ground reaction force vector passes through the CoM
- **Obs3:** LIP Dynamics (1) is **unstable**

Interpretation: $\ddot{c}_x$ is given by a repulsive force pushing away c_x from z_{xy} (e.g. always tipping over unless z_{xy} is under c_{xy})



LIP interpretation in legged robots

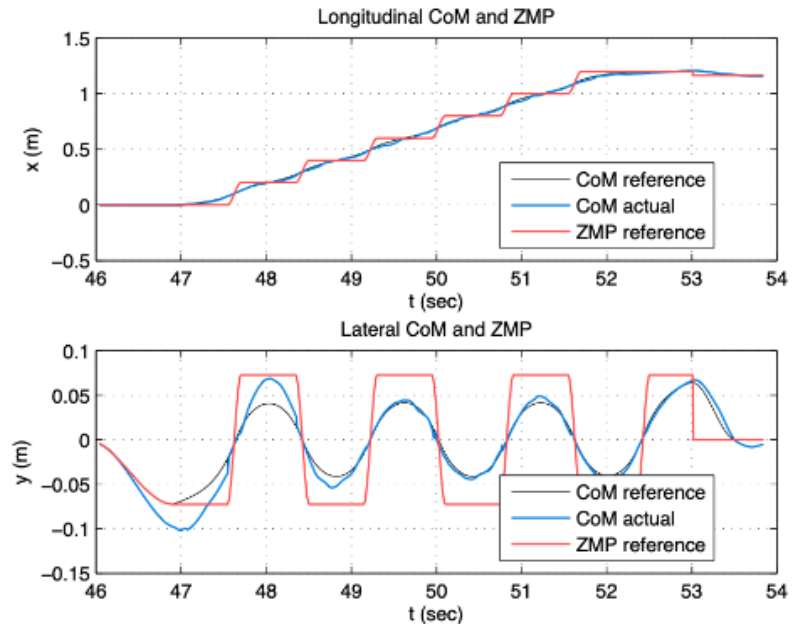
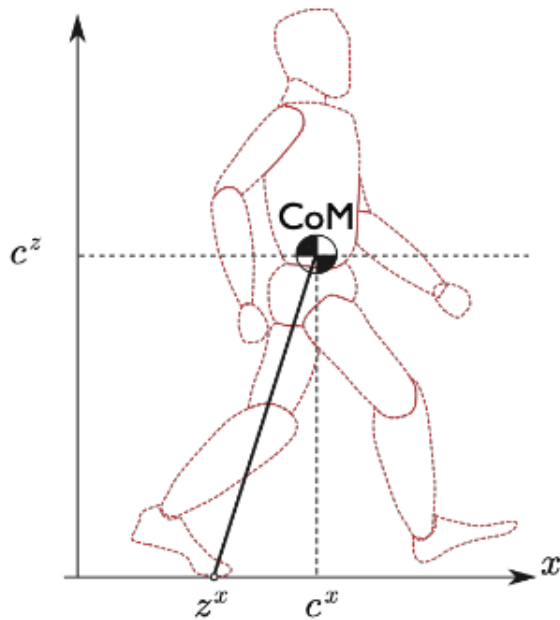
$$\ddot{c}_{xy} = \frac{g}{c_z} (c_{xy} - z_{xy})$$

$$\ddot{c}_x = \frac{g}{c_z} (c_x - z_x)$$

$$\ddot{c}_y = \frac{g}{c_z} (c_y - z_y)$$

Longitudinal direction

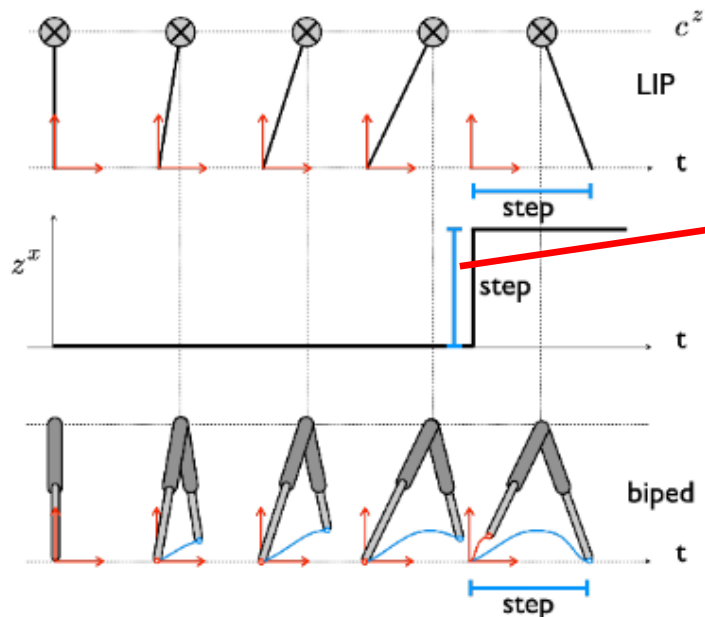
Lateral direction



- the simplest interpretation of the LIP is that of a telescopic (so to remain at a constant height) massless leg in contact with the ground at ZMP

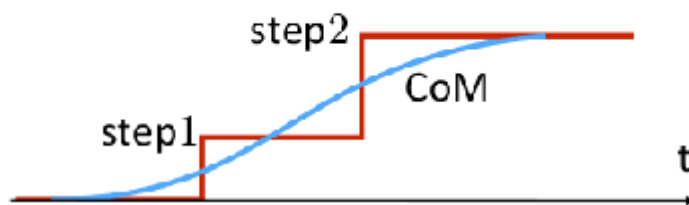
LIP interpretation in legged robots: Longitudinal

- Robot = compass biped with only one leg touching the ground at the same time

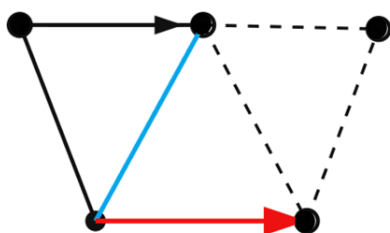


Discontinuity due to the change of reference frame (Hp instantaneous change)

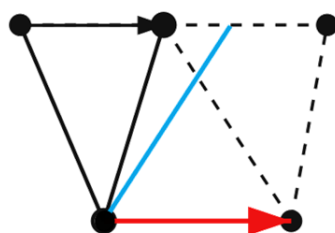
$$p^x = z^x \quad \text{— red —} \quad c^x \quad \text{— blue —}$$



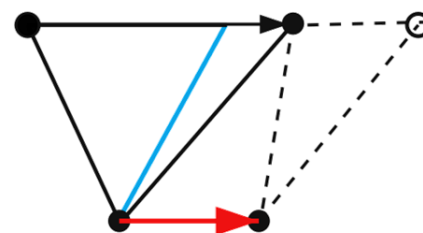
- Point foot assumption: the ZMP changes discontinuously with stepping
- The motion of the LIP is determined only by the initial condition, but this can be changed at each TOUCH-DOWN. $\Rightarrow$ Changing STEP TIMING we can accelerate/decelerate.



SYMMETRY



EARLY TD (SLOW DOWN)

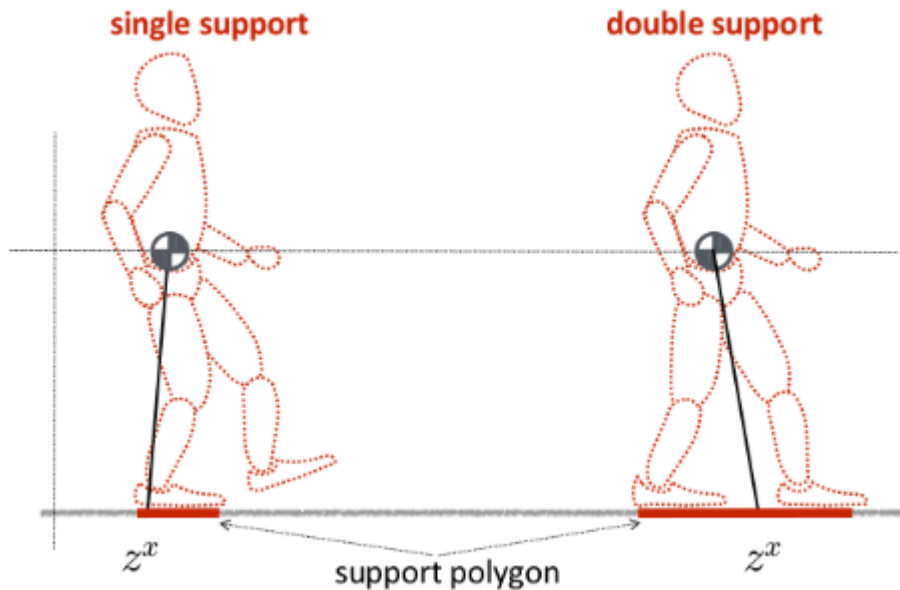


DELAYED TD (ACCELERATE)

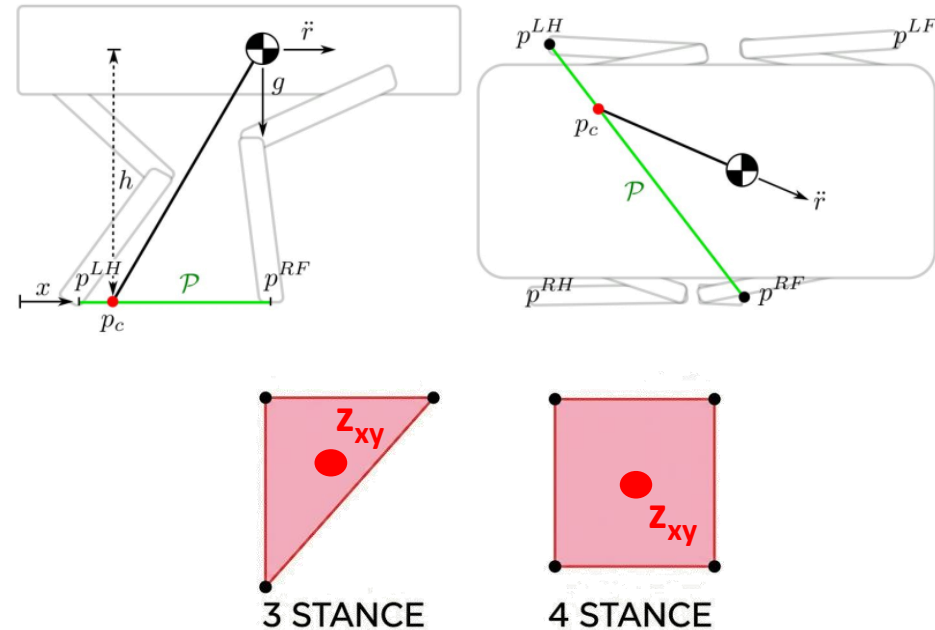
Finite size foot/ support polygon

- Extension considering single and double support phase
- with different loading configurations we can balance by putting ZMP UNDER CoM.

BIPED



QUADRUPED



- we can change ZMP without stepping! By changing loading condition we shift the ZMP

CoP and contact forces

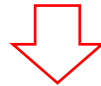
- contact forces with the environment are physically limited (unilateral contact) ($f_{i,z} > 0$)

$\oplus$

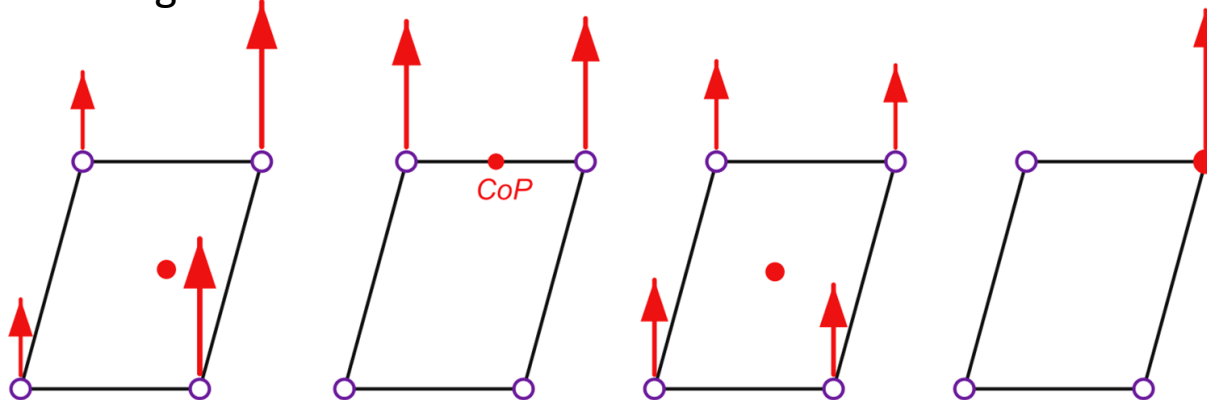
$$f_z = \sum_{i=1}^c f_{z_i} : \text{resultant of contact forces}$$

- CoP is a convex combination of feet forces with positive weights

$$\text{cop} = \frac{\sum_{i=1}^c p_i f_{z_i}}{\sum f_{z_i}} \rightarrow \sum_{i=1}^c p_i \alpha_i \quad \begin{cases} \alpha_i > 0 \\ \sum \alpha_i = 1 \end{cases}$$



- CoP will always lie in the convex hull of the supporting feet** (support polygon), valid only for flat ground



- CoP cannot exist out of the polygon (unless the the feet can “pull” on the ground)

Zero Moment Point

- The CoP z_{xy} is also known as **Zero Moment Point (ZMP)**

- ZMP criterion of dynamic balance:**

Exists control authority

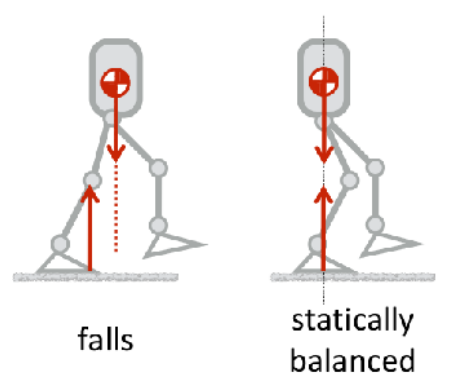


ZMP inside
support polygon



Exists a set of **feasible** contact force (resultant)
able to counterbalance both inertial and gravity
force (avoiding tipping over) (**Dynamic Balance**)

- Static Balance:** Ground reaction forces counteract gravity only (no motion)

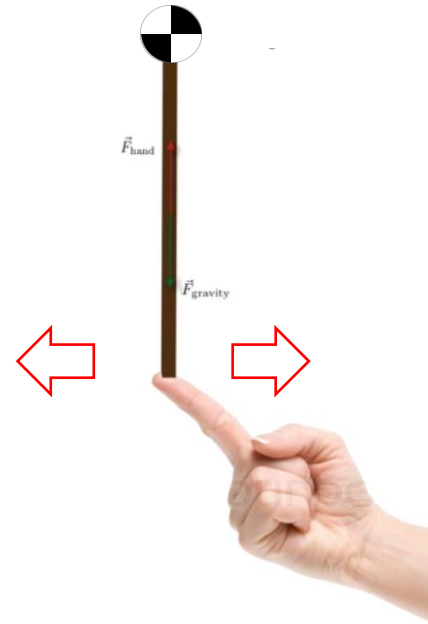


- Static balance criterion:** CoM inside the support polygon

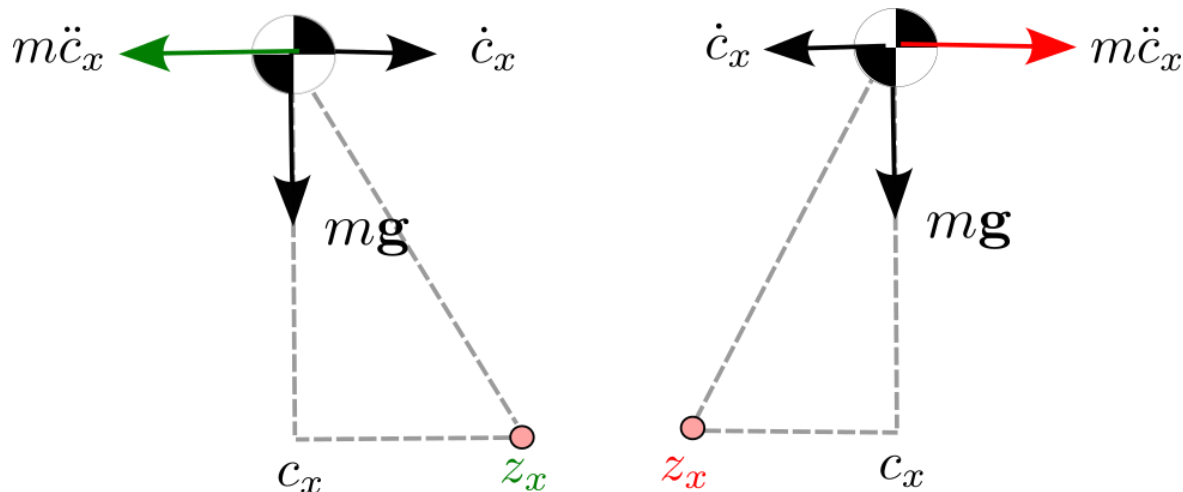
Balance strategy

- We can shift the CoP under the CoM to have $\ddot{c}_{xy} = 0$

$$\ddot{c}_{xy} = 0 \iff c_{xy} = z_{xy} \quad \Rightarrow \quad \text{ZMP coincides with the CoM projection on ground}$$

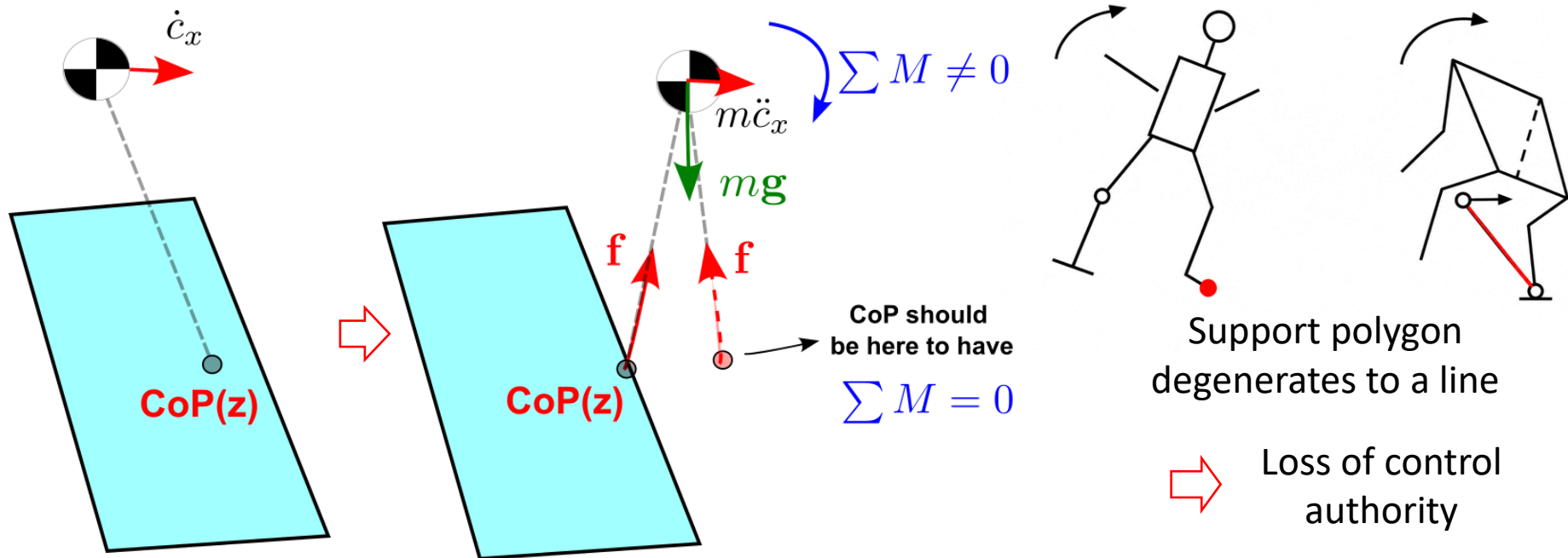


- We can create **forward/backward** acceleration by moving the Cop/ZMP **backward/forward** w.r.t. The CoM (eg to exploit gravity)
- The position where the ZMP is located with respect to gravity creates the motion



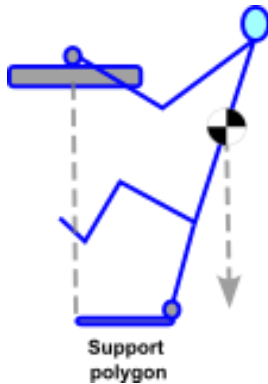
Tipping over

- Let's assume we want to decelerate the COM motion $\rightarrow$ we place the ZMP in front of COM
- Assume the deceleration is not enough to stop the motion
- When the ZMP reaches the boundary it cannot move beyond and there is no longer equilibrium between inertial and gravity forces and the robot starts to rotate about the edge



Summary of ZMP

- Synthesizes GRF distribution in a single point
- if ZMP is inside support polygon the motion is feasible with the available contact forces (**ZMP CRITERION**) and the robot is dynamically balanced
- with no motion (STATIC), ZMP criterion becomes **COM CRITERION**
- defined for coplanar contacts
- for non-coplanar contacts ZMP/CoM criteria cannot discriminate if a system is dynamically/statically balanced



Example of Static balance with CoM
out of support polygon

- having the ZMP inside polygon does not ensures that the robot is not falling (see capture point) but that exists contact forces consistent with the motion

Summary of LIP model

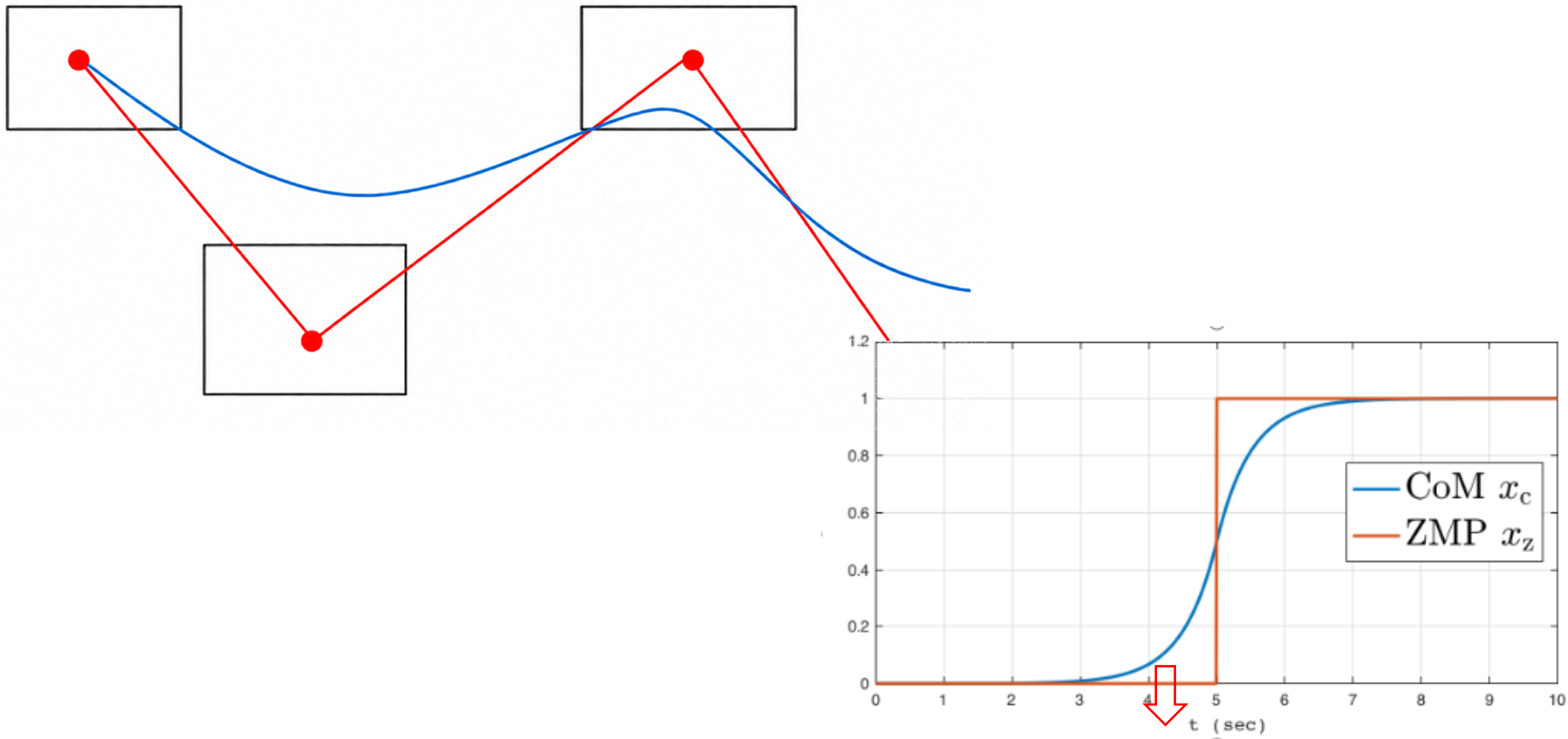
- Simplifying assumptions:
 - $c_z = \text{const}$
 - no angular dynamics
 - coplanar feet
- Dynamics is:
 - linear
 - decoupled for x, y directions
 - tells you how CoM will move w.r.t ZMP (that is computed from contact points)



I can design COM trajectory such that the correspondent ZMP satisfies dynamic balance (ZMP CRITERION)

Walking pattern generator based on LIP

- ① Specify a target **ZMP trajectory** (e.g., from footholds) that satisfies the ZMP criterion.
- ② Compute an appropriate **CoM motion** that satisfies Equation (1).



CoM needs to start moving beforehand. Preview (receding horizon) generates the anticipative action.

Examples planning with LIP

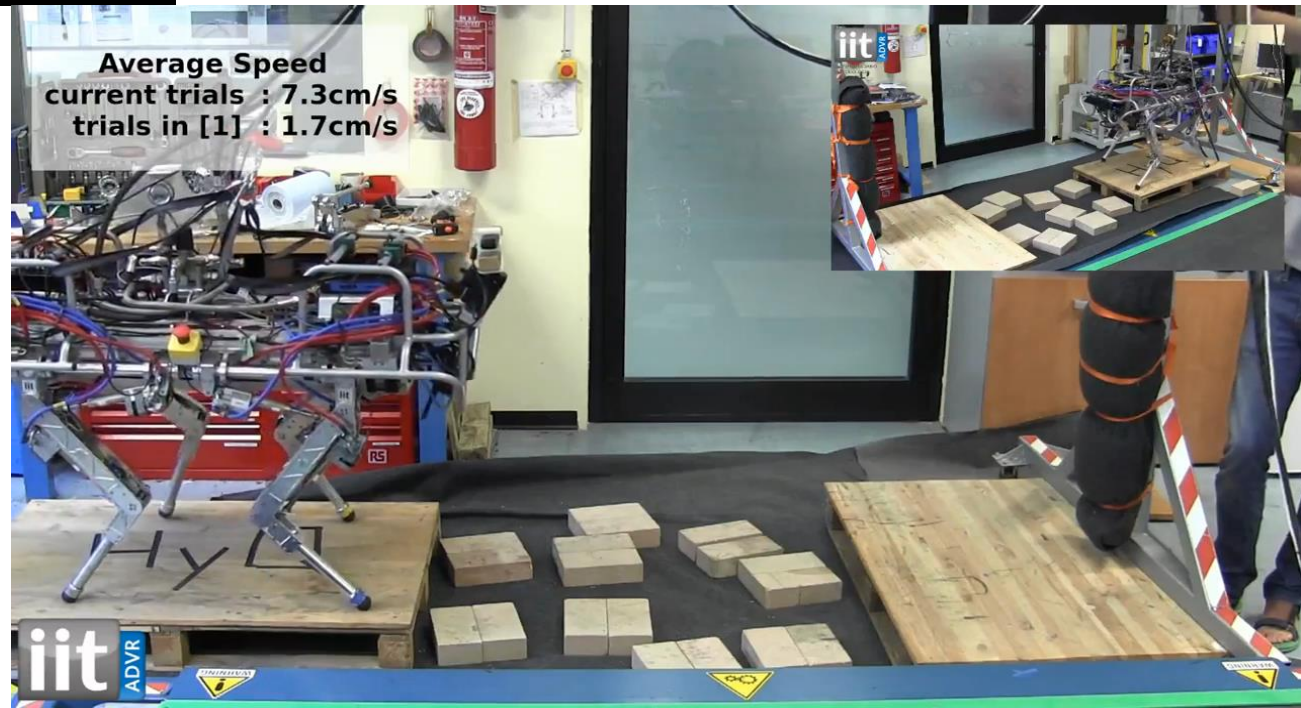
LittleDog at MIT



Gray Modular Boards: 10.3 cm/s

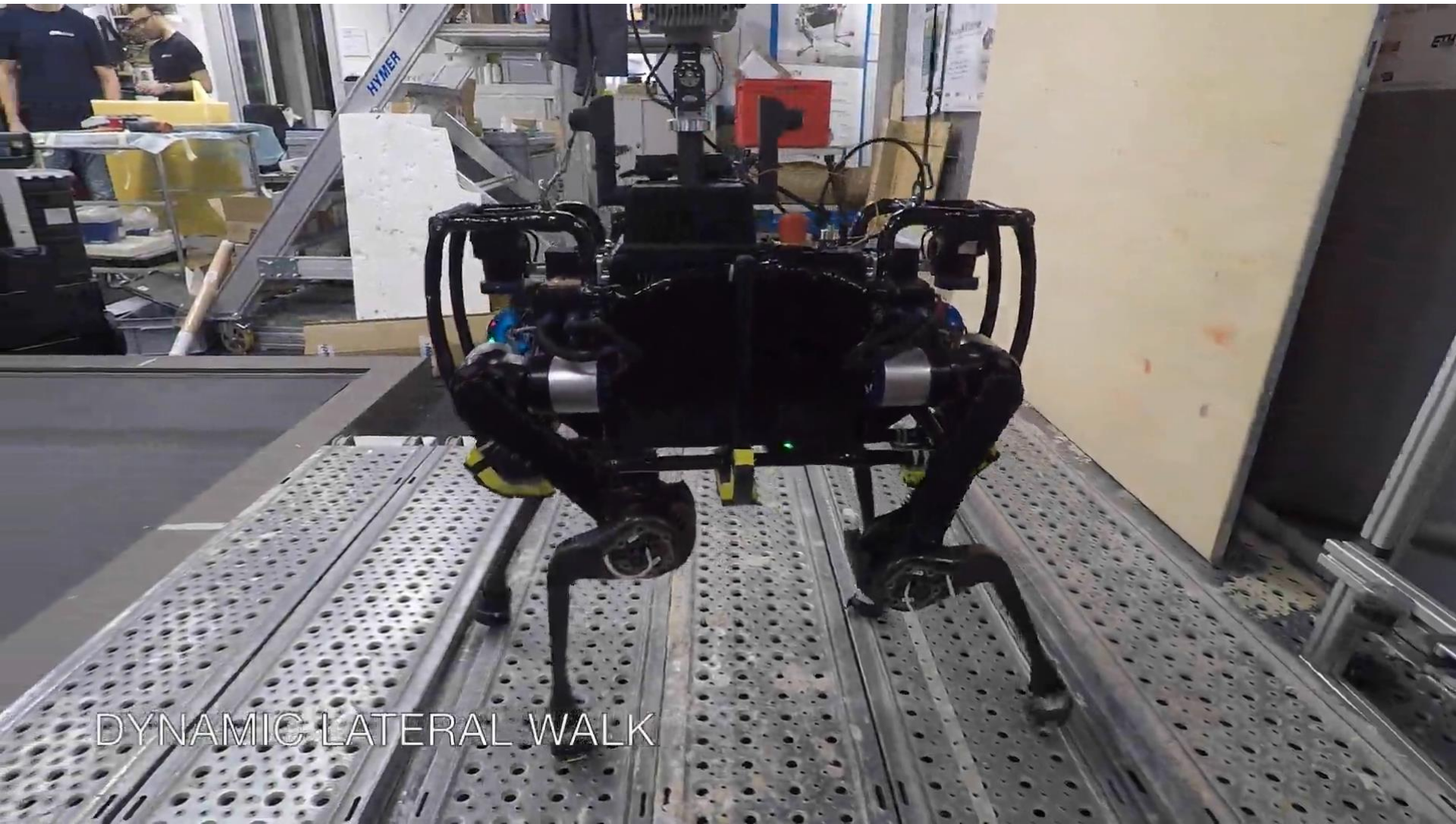
[Learning, planning, and control for quadruped locomotion over challenging terrain, Kalakrishnan et al. **2010**]

[Winkler et al. Planning and Execution of Dynamic Whole-Body Locomotion for a Hydraulic Quadruped on Challenging Terrain, **2015**]



Examples planning with LIP

[Bellicoso et al. Dynamic Locomotion and Whole-Body Control for Quadrupedal Robots, 2017]



DYNAMIC LATERAL WALK

Integration of LIPM Dynamics

$$\ddot{c}_x = \underbrace{\frac{g}{c_z}}_{\omega^2} (c_x - z_x) \quad \text{defining} \quad \omega = \sqrt{\frac{g}{c_z}}$$

- Rewrite in state-space form

$$\underbrace{\frac{d}{dt} \begin{bmatrix} c_x \\ \dot{c}_x \end{bmatrix}}_{\dot{x}} = \underbrace{\begin{bmatrix} 0 & 1 \\ \omega^2 & 0 \end{bmatrix}}_A \underbrace{\begin{bmatrix} c_x \\ \dot{c}_x \end{bmatrix}}_x + \underbrace{\begin{bmatrix} 0 \\ -\omega^2 \end{bmatrix}}_B \underbrace{z_x}_u$$

- Eigenvalues

$$\lambda^2 c_x = \omega^2 c_x \Rightarrow (\lambda^2 - \omega^2) c_x = 0 \Rightarrow \lambda^2 = \omega^2,$$

$$\lambda_{1,2} = \pm \omega \quad (1 \text{ stable} / 1 \text{ unstable}). \quad \Rightarrow \text{2 real eigen values, 1 positive, Unstable dyn.}$$

- Being a **linear** system in this case a closed form (exact) solution exists to the integration of the ODE

$$x(t) = \underbrace{C_1 e^{\omega t} + C_2 e^{-\omega t}}_{\text{homogeneous}} + \underbrace{u}_{\text{particular}}$$

Integration of the ODE

- Imposing initial conditions and exploiting

$$\cosh(x) = \frac{e^x + e^{-x}}{2}$$

$$\sinh(x) = \frac{e^x - e^{-x}}{2}$$

$$\begin{cases} x(t) = x_0 \cosh(\omega t) + \frac{\dot{x}_0}{\omega} \sinh(\omega t) + u - u \cosh(\omega t) \\ \dot{x}(t) = x_0 \omega \sinh(\omega t) + \frac{\dot{x}_0}{\omega} \omega \cosh(\omega t) - u \omega \sinh(\omega t) \end{cases}$$

- Rewriting in matrix form:

$$\begin{bmatrix} c_x(t) \\ \dot{c}_x(t) \end{bmatrix} = \begin{bmatrix} \cosh(\omega t) & \frac{1}{\omega} \sinh(\omega t) \\ \omega \sinh(\omega t) & \cosh(\omega t) \end{bmatrix} \begin{bmatrix} c_{x_0} \\ c_{\dot{x}_0} \end{bmatrix} + \begin{bmatrix} 1 - \cosh(\omega t) \\ -\omega \sinh(\omega t) \end{bmatrix} z_x$$

- One-step discrete integration

$$\begin{bmatrix} c_x(k+1) \\ \dot{c}_x(k+1) \end{bmatrix} = \begin{bmatrix} \cosh(\omega dt) & \frac{1}{\omega} \sinh(\omega dt) \\ \omega \sinh(\omega dt) & \cosh(\omega dt) \end{bmatrix} \begin{bmatrix} c_x(k) \\ c_{\dot{x}}(k) \end{bmatrix} + \begin{bmatrix} 1 - \cosh(\omega dt) \\ -\omega \sinh(\omega dt) \end{bmatrix} z_x(k)$$

- we can integrate and predict the CoM trajectory along an horizon

Outline

Limits of Instantaneous Control

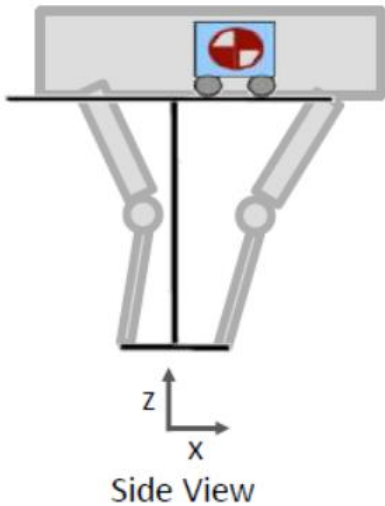
Single Rigid Body Model

Centroidal Dynamics Model

Linear Inverted Pendulum Model (LIPM)

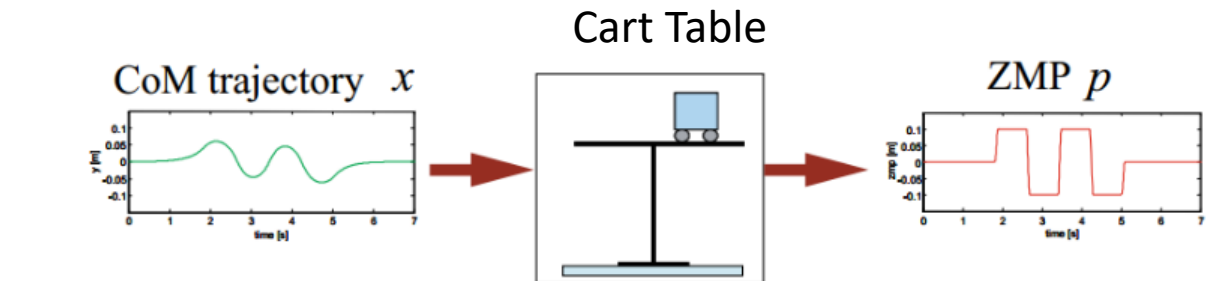
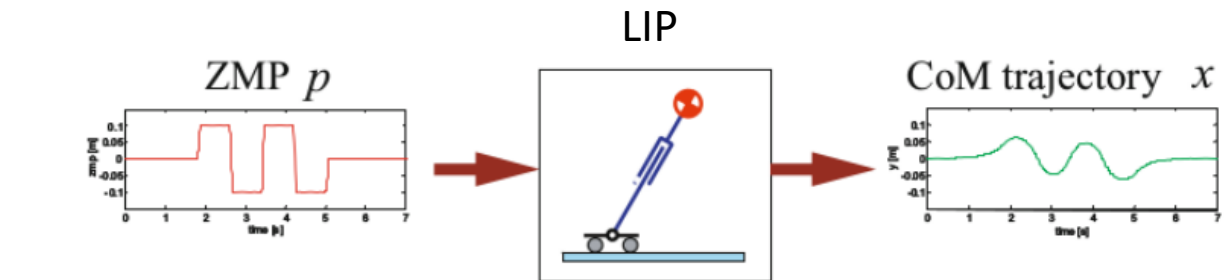
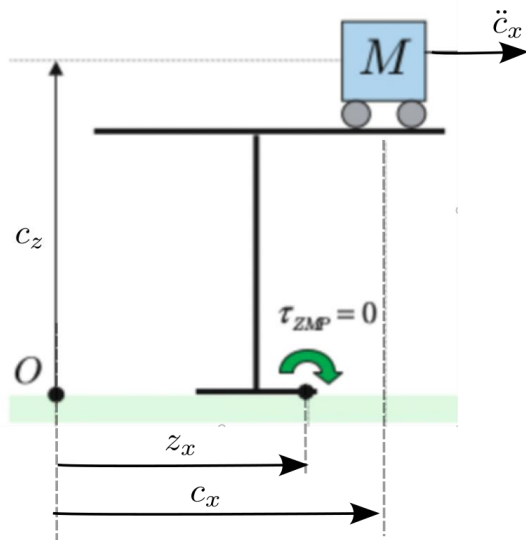
Other Models

Cart-table model



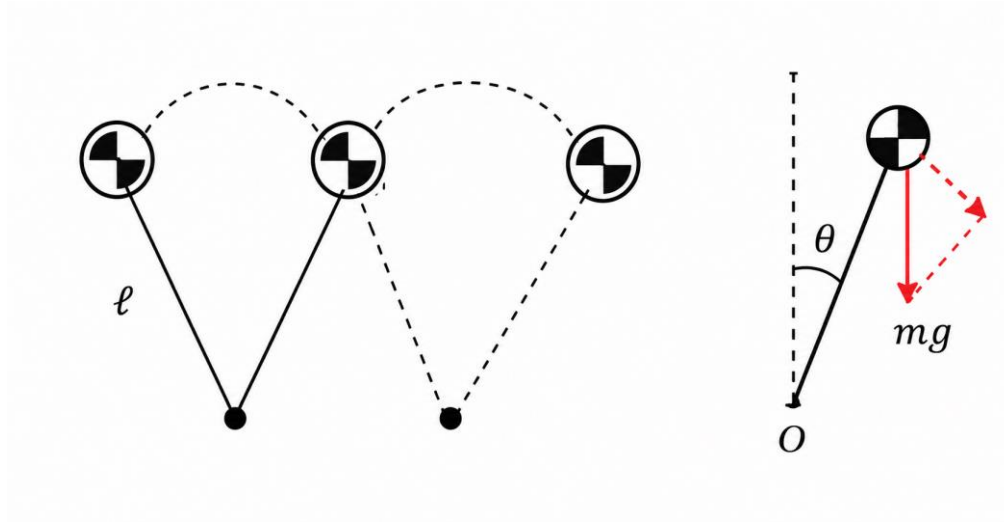
- The robot is modeled like a cart-table
- Even if the foot is small, having a cart accelerating on the table can still keep an instantaneous balance
- Same equation as LIP with input/output flipped

$$z = c_x - \frac{c_z}{g} \ddot{c}_x$$



(b)

Inverted pendulum



$$\cancel{ml^2} \ddot{\theta} = \cancel{mgl} \sin \theta$$



$$\ddot{\theta} = \frac{g}{l} \sin \theta$$

- Non linearized version of LIP

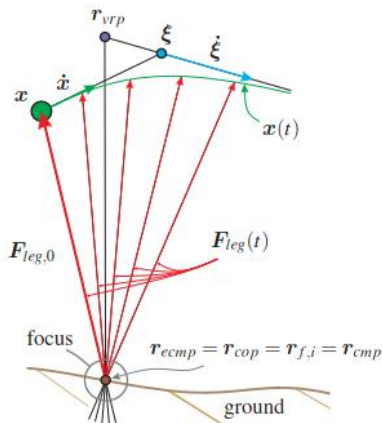
Point mass Model

$$m\ddot{c} = \underbrace{\sum_i f_i}_{F_R} + mg$$

- Robot is modelled as a point mass, 3D translational states/DoFs
- Neglects angular dynamics: good assumption for humanoids moving slow where angular momentum is quite low
- good choice for walking motions

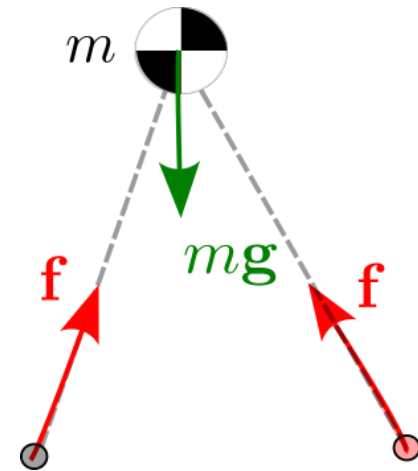
✓ Small state (3D), **linear** dynamics

✓ It can handle 3D motions! including rough-terrain applications

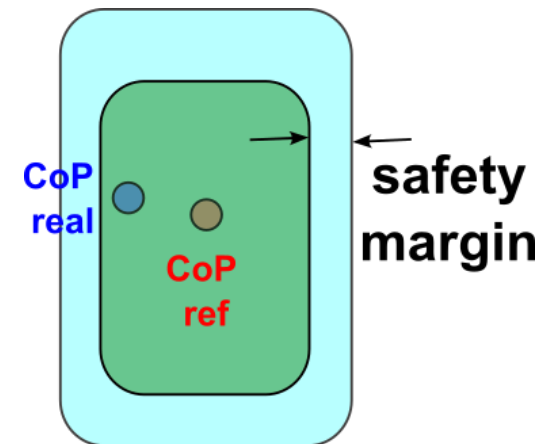


[Englsberger et al., 2013]

[wieber 2008]

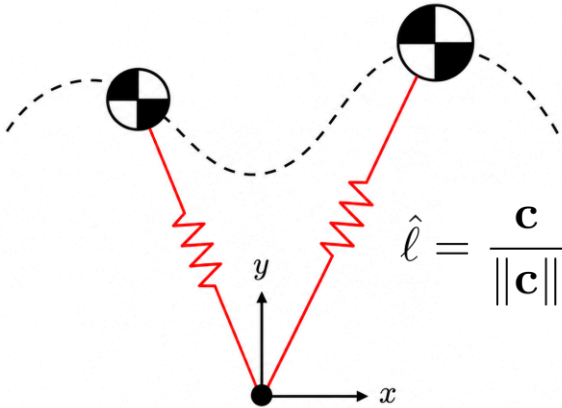


Difference with Real Center of Pressure appears to be less than 2 cm; can be easily taken into account by introducing proper safety margins



Spring loaded inverted pendulum – SLIP

- Used for running



2 PHASES:

- ① **Flight Phase:** exchange of kinetic/grav. potential energy
- ② **Stance Phase:** exchange of kinetic/grav. potential energy/elastic potential energy

Flight phase equation

$$\ddot{\mathbf{c}} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix}$$

Stance phase equation

$$m\ddot{\mathbf{c}} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + \hat{\ell} f(\|\mathbf{c}\|, \ell_0)$$

Hook's law

$$f(\|\mathbf{c}\|, \ell_0) = k(\ell_0 - \|\mathbf{c}\|)$$

Why Springs? We store energy in tendons

Models mostly used in legged locomotion

Model	# DOFs	Flight	Non- flat terrain	Angular dyn.	Friction constr.	Torque constr.	Kin. con- str.	Input
LIP (walking)	2	×	×	×	×	×	×	z
LIP + Flywheel	2	×	×	✓	×	×	×	z
SLIP (running)	3	✓	✓	×	×	×	×	F_R
Point Mass	3	×	✓	×	✓	×	×	F_R
Centroidal Dynamics	6	✓	✓	✓	✓	×	×	f_i
Centroidal Dyn.+ Kinematics	$6 + n$	✓	✓	✓	✓	×	✓	f_i, q_J
Full Dynamics	$6 + n$	✓	✓	✓	✓	✓	✓	τ